Lesson notes · for you

The agent loop

AI agents · Upper secondary · Lesson 3

Time
20 min

Materials
Your phone or a school computer, for the code steps; Paper, a pen and three small cards

Learning goals

- Write the agent loop in Python.
- Stop a stuck model with a step limit.
- Test a loop honestly with a scripted model.

Activity: Be the loop

1. Set up (3 min): In threes, write one card per role: the model (with a script of lines to read), the tool (answers for weather and route in Kafi and Dala) and the loop (a step counter).
2. Play (7 min): The loop says “your turn” to the model, who reads the next line of the script. If it’s a call, the loop passes it to the tool, writes the call and the result on the history sheet, and asks the model again. If it’s an answer, stop.
3. Stuck (5 min): Write a script with only calls. The loop counts the steps aloud and stops at four, then says “Stopped: I couldn’t finish this.”
4. Code (5 min): In the code step, turn what the loop did on paper into a for loop with a limit, and run the tests.

Write two script cards before you start: one that answers after a call, and one that never answers. No devices are needed until the last part.

Discussion

- Why is while True risky in an agent?
- What should the person be told when the loop stops?
- What could a real model do that your script never did?

Check yourself

- Write the four steps of the loop from memory, and where the step limit goes.

Try it this week
Play the loop with two friends: one is the model, reading from a script card; one is the tool; you are the loop. Count the steps and stop at four. Then try a script that never answers, and see who stops it.

Worksheet

The agent loop

AI agents · Upper secondary · Lesson 3

Name _____ Class _____ Date _____

1 **Your loop passes every test with a scripted model. What have you shown?**

- ◯ That your loop works for the outputs you tested
- ◯ That every tool gives the right result
- ◯ That the real model will behave the same

2 **Put one turn of the agent loop in order.**

- ☐ Add the call and its result to the history
- ☐ Otherwise, split the call into a name and details
- ☐ Ask the model for its next output
- ☐ If it has no curly bracket, return it as the answer
- ☐ Go round again, up to the step limit
- ☐ Run the tool

3 **This agent stops after one output and has no limit. Fix agent(request, limit): loop up to limit steps; return the first answer (text with no curly bracket); add each call and its tool's result to history; and return "Stopped" when the steps run out. The model here is scripted: SCRIPT is what it writes.**

```
SCRIPT = ['weather{"place":"Kafi"}',
          "Sunny in Kafi."]

def model(history):  # scripted
    return SCRIPT[len(history) // 2]

def tool(call):  # a pretend tool
    return '{"sky":"sunny"}'

def agent(request, limit=4):
    history = [request]
    out = model(history)
    return out
```

Write your program:

4 **This agent can run for ever if the model never answers. Which line causes it?**

Circle the line with the mistake. Write it correctly:

```
1  def agent(request):
2      history = [request]
3      while True:
4          out = model(history)
5          if "{" not in out:
6              return out
7          history += [out, tool(out)]
```

---

5 **What does this program print?**

```
SCRIPT = ["a", "b", "c"]
history = ["q", "a", "r"]
print(SCRIPT[len(history) // 2])
```

◯ a ◯ b ◯ c

✎ **The loop, not the model, decides when the agent stops. Where else do we set a limit instead of trusting something to stop by itself?**

Check your answers

The agent loop

1 **That your loop works for the outputs you tested**

A script tests your code. A real model writes new things, wrong calls and stuck loops included, so your code must handle those too.

2 **Top to bottom: 5, 3, 1, 2, 6, 4**

The model writes first; an answer ends the loop at once.

3

```
SCRIPT = ['weather{"place":"Kafi"}',
          "Sunny in Kafi."]

def model(history):  # scripted
    return SCRIPT[len(history) // 2]

def tool(call):  # a pretend tool
    return '{"sky":"sunny"}'

def agent(request, limit=4):
    history = [request]
    for step in range(limit):
        out = model(history)
        if "{" not in out:
            return out
        history += [out, tool(out)]
    return "Stopped"
```

The for loop gives the model at most limit turns.

4 **Line 3: `for step in range(4):`**

while True only ends when the model answers.

5 **b**

len(history) is 3 and 3 // 2 is 1, so the scripted model returns SCRIPT[1], its second line: one call and one result have been added since the request.